

Test Queries Report

RAG AI Assistant — "Attention Is All You Need" Telegram Bot (n8n + Pinecone + Gemini)

This report documents five end-to-end test queries executed against the deployed Telegram bot. Each query was sent as a live Telegram message to the running Query workflow, which embeds the question, retrieves the top matching passages from Pinecone (Top K = 5, cosine similarity), and generates a strictly retrieval-grounded answer. Each test below shows the exact input sent, the assistant's classification of whether relevant content existed, the full response delivered back to the user on Telegram, and the source citation returned with the answer.

Summary

#	Input Query	Query Type	Result	Status
1	What is scaled dot-product attention and how is it computed?	Content Question	Answered with citation	SUCCESS
2	What BLEU score did the Transformer (big) model achieve on the English-to-German translation task?	Content Question	Answered with citation	SUCCESS
3	Why did the authors choose self-attention over recurrent and convolutional layers?	Content Question	Answered with citation	SUCCESS
4	How many attention heads does the base Transformer model use?	Content Question	Answered with citation	SUCCESS
5	What is the best place in Germany?	Fallback Check (out-of-scope)	Correct fallback returned	SUCCESS

All five queries were routed through the same single RAG pipeline — there is no keyword matching or manual routing involved. Whether a question is answered with cited content or met with the fallback message is determined entirely by Pinecone's similarity search results and the AI Agent's own judgment of relevance, as instructed by its system prompt.

Detailed Input / Output Pairs

Test 1

Type: Content Question | Status: SUCCESS

Input (sent via Telegram):

What is scaled dot-product attention and how is it computed?

Output (bot reply):

Scaled dot-product attention takes as input queries and keys of dimension d_k , and values of dimension d_v . The attention function is computed as the weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key. The output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) V$$

The two most commonly used attention functions are additive attention and dot-product (multiplicative) attention. Dot-product attention is identical to the algorithm described above, except for the scaling factor of $1/\sqrt{d_k}$. While additive attention computes the compatibility function using a feed-forward network with a single hidden layer, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

Sources:

Attention Is All You Need | Chunk #5 | Page 3

Test 2

Type: Content Question | Status: SUCCESS

Input (sent via Telegram):

What BLEU score did the Transformer (big) model achieve on the English-to-German translation task?

Output (bot reply):

The Transformer (big) model achieved a BLEU score of 28.4 on the English-to-German translation task.

Sources:

Attention Is All You Need | Chunk #15 | Page 9

Test 3

Type: Content Question | Status: SUCCESS

Input (sent via Telegram):

Why did the authors choose self-attention over recurrent and convolutional layers?

Output (bot reply):

The authors chose self-attention over recurrent and convolutional layers to satisfy three specific criteria:

1. **Computational Complexity:** Self-attention layers are faster than recurrent layers when the sequence length n is smaller than the representation dimensionality d , and they provide a more efficient way to connect all positions in a sequence compared to convolutional layers.
2. **Parallelization:** Recurrent layers are inherently sequential, requiring $O(n)$ operations, which precludes parallelization within training examples. Self-attention connects all positions with a constant number of sequentially executed operations, allowing for significantly more parallelization.
3. **Path Length for Long-Range Dependencies:** Learning long-range dependencies is easier with shorter paths between positions. Self-attention reduces the path length between any two positions in the input and output sequences to a constant number of operations, whereas convolutional layers require a stack of $O(n/k)$ or $O(\log_k(n))$ layers, and recurrent layers require $O(n)$ operations to relate signals between distant positions.

Additionally, the authors noted that self-attention could yield more interpretable models.

Sources:

Attention Is All You Need | Chunk #12 | Page 6

Note: this question initially returned the fallback message on a first attempt phrased conversationally ("why did the authors choose..."), since that exact wording does not appear in the paper. Retrieval succeeded once topK was increased and the system prompt was updated to encourage rephrasing before concluding no relevant content exists — see the Design Write-Up for details on this retrieval-accuracy limitation and fix.

Test 4

Type: Content Question | Status: SUCCESS

Input (sent via Telegram):

How many attention heads does the base Transformer model use?

Output (bot reply):

The base Transformer model uses 8 attention heads.

Sources:

Attention Is All You Need | Chunk #12 | Page 5

Test 5

Type: Fallback Check (out-of-scope) | Status: SUCCESS

Input (sent via Telegram):

What is the best place in Germany?

Output (bot reply):

I could not find relevant information in the knowledge base.
